Hochschule Karlsruhe
University of Applied Sciences

Fakultät für **Informatik**
und **Wirtschaftsinformatik**

Dipl. WiInf. Lars Thielemann
Prof. Dr. Heiko Körner

🧪 Tag 3 – Inventar-App

Ziel der Übung

Erstelle eine kleine **Inventar-App** (Department, Person, Device, Assignment), bei der

- das **DB-Schema** über ein **Init-SQL** im Projekt definiert wird,
- zentrale **Domänenregeln** (z. B. max. eine aktive Ausleihe) technisch umgesetzt sind,
- du beim Vorführen **Branch-Wechsel** und wichtige Teile der App **erklären** kannst.

Ausgangsmaterial

Für die Übung nutzt ihr als **Arbeitsbasis** den **inventar-starter**, der auf der Vorarbeit von Tag 2 aufsetzt.

Als **Referenzprojekt** steht euch der **grades-starter** zur Verfügung.

Wichtig: Du arbeitest im **inventar-starter**!

Abgabe

Ihr zeigt in der Gruppe:

1. **Lauffähige Inventar-App**
   - Health (`/health`) und API-Doku (`/docs`) funktionieren.
2. **Inventarverwaltung**
   - Inventar-Schema liegt als **Init-SQL** im Projekt (z. B. `db/init/010_inventory_schema.sql`).
   - CRUD für Devices
   - Erstellen und Auflisten von Assigments
   - Rückgabe von Assigments
   - Domänenregeln werden eingehalten
3. **MQTT**
   - MQTT-Events für Ausleihen.

- MQTT-Chat zwischen zwei Terminals (oder zwei Personen) mit `mosquitto_pub` / `mosquitto_sub`.

Voraussetzungen

- `inventar-starter.zip` als lauffähige Arbeitsbasis mit Docker, Postgres, MQTT, /health und /docs, basierend auf dem Repo von Tag 2.
- `grades-starter.zip` als Blaupause für technische Muster , z. B.:
  * Struktur von backend/app
  * SQL-Initialisierung
  * Form-Verarbeitung mit FastAPI
  * Templates mit Jinja
  * Teil-Updates mit htmx
  * einfache Pydantic-Modelle

A) Inventar-Branch für Tag 3 anlegen

`Zeitbudget: ca. 15–20 min`

Ziel: Ihr legt einen sauberen, gemeinsamen Ausgangspunkt für Tag 3 an. n.

1. Wechselt zunächst in eurem Repository auf main.

2. Prüft kurz euren derzeitigen Arbeitsstand:
   - Gibt es noch nicht commitete Änderungen?
   - Falls ja: erst committen, stashen oder bewusst sichern.
3. Übernehmt anschließend den bereitgestellten **inventar-starter** in euer Repository. Dieser Starter ist ab jetzt eure Arbeitsbasis für Tag 3.

4. Startet das Projekt und prüft es auf Lauffähigkeit:
   - `docker compose up -d --build`
   - Prüfe:
     * `http://localhost:8000/health`
     * `http://localhost:8000/docs`
5. Commitet diesen Stand zunächst auf main, z. B. mit der Nachricht `Startpunkt Tag 3: inventar-starter.`

Wichtig: Für Tag 3 ist der **inventar-starter** euer neuer gemeinsamer Ausgangspunkt. Erst nach diesem Basis-Commit auf main arbeitet ihr in den Folgeaufgaben in Feature-Branches weiter.

B) Referenzprojekt grades-starter verstehen

`Zeitbudget: ca. 20–30 Min.`

Ziel: Ihr schaut euch technische Muster an, die ihr später für eure Inventar-App übernehmen könnt.

1. **grades-starter separat öffnen**
   - `grades-starter.zip` entpacken (z. B. in `grades-starter/`).
   - Der-Container bauen und starten
   - Prüfen:
     * `http://localhost:8000/health`
     * `http://localhost:8000/docs`
     * UI (z. B. `/grades`, `/students`).
2. **Schaut euch gezielt folgende Stellen an:**
   - **DB-Init**: `db/init/001_schema.sql`
     * Welche Tabellen gibt es (`student`, `module`, `grade`)?
     * Wo stehen **Constraints** (UNIQUE, CHECK, FK)?
   - **DB-Zugriff**: `backend/app/db.py`
     * Wie wird `get_conn()` verwendet?
   - **API**:
     * Wo werden Routen für Students / Grades definiert?
   - **Models**: `backend/app/models.py`
     * Wie werden Models aufgebaut?
   - **UI**:
     * Templates unter `backend/app/templates/…` (Listen, Formulare).
3. **Klärt und macht euch Notizen** (z. B. `docs/grades-notes.md`)
   - Wie ist das Projekt strukturiert?
   - Wie werden Formulare verarbeitet?
   - Wie weden Templates eingebunden?
   - Wie sehen einfache CRUD-Flows aus?

Wichtig: Ihr entwickelt nicht im grades-starter weiter. Der **grades-starter** dient nur als Referenzprojekt für Struktur, Formulare, Templates und CRUD-Muster.

C) Feature-Branch für die Inventar-App anlegen

`Zeitbudget: ca. 10–15 Min.`

Ziel: Ihr beginnt die eigentliche Umsetzung der Inventar-App auf Basis eures neuen Tag-3-Startpunkts.

1. **Zurück in euer Inventar-App**

- Geht zurück in euer Inventar-Projekt
- Sicherstellen, dass `main` auf dem soeben erzeugten Stand ist

2. **Neuer Branch für die Inventar-App**
   - z.B. als `feat/inventar-app`
   - (Optional) direkt pushen
3. **Arbeiten im Feature-Branch**

   Wie üblich in Branches:
   - fachliche oder technische Änderungen in einem Feature-Branch,
   - sinnvolle Commits,
   - anschließend zurück nach `main`

D) Inventar-Schema als Init-SQL

`Zeitbudget: ca. 30 min`

Ziel: DB-Schema aufbauen.

Wichtig: Keine Tabellen per Hand in der DB anlegen**, sondern über eine** Init-SQL-Datei im Projekt**.**

1. **Schemata für die Inventar-Domäne anlegen**
   - department
   - person
   - device_type
   - location
   - device
   - assignment
2. **Achte auf:**
   - Primärschlüssel
   - Fremdschlüssel
   - sinnvolle NOT NULL-Constraints
   - eindeutige Seriennummern für Devices
3. **Einfache Seed-Daten**
   - 2–3 `device_type'` (z. B. „Laptop“, „Monitor“, „Handscanner“).
   - 2-3 `location` (z.B. „Gebäude E“, „Gebäude F“, „Gebäude H“).
   - Eine Handvoll `person`-, `device` und `assigment`-Einträge.
4. **Compose neu starten & Schema prüfen**
   - `docker compose down`
   - `docker compose up -d --build`
   - Mit psql im DB-Container nachsehen:

```
docker compose exec db psql -U appuser -d appdb -c "\dt"
docker compose exec db psql -U appuser -d appdb -c "\d device"
# oder mit
docker compose exec db psql -U appuser -d appdb
\dt
\d device
select * from device;
\q
```

- Ggf. Fehler im Init-SQL korrigieren und Container neu starten.

5. **Commit**
   - Änderungen adden und committen.

E) API & Domänenregeln umsetzen

`Zeitbudget: ca. 45–60 min`

Ziel: Ihr baut die ersten fachlichen Endpunkte für eure Inventar-App.

1. Ergänzt in der API erste **Endpunkte für Devices**, z. B.:
   - **GET** `/devices`
   - **POST** `/devices`
2. Ergänzt anschließend **Endpunkte für Assignments**, z. B.:
   - **GET** `/assignments/active`
   - **POST** `/assignments`
   - **POST** `/assignments/{id}/return`
3. Setzt dabei mindestens folgende **Domänenregeln** um:
   - **IR-01**: `serial_number` ist eindeutig
   - **IR-02**: Ein Device darf maximal eine aktive Ausleihe haben
   - **IR-03**: `returned_at` darf nicht vor `issued_at` liegen
4. **Reagiert bei Fehlern** mit sinnvollen Statuscodes, z. B.:
   - `409` Conflict bei doppelter Seriennummer
   - `409` Conflict bei zweiter aktiver Ausleihe
   - `422` bei ungültiger Zeitlogik
5. **Testet** die Endpunkte über `/docs`.

Der grades-starter kann euch hier als Referenz helfen:

- Wie werden Formulardaten verarbeitet?
- Wie werden DB-Zugriffe strukturiert?
- Wie sehen einfache CRUD-Flows aus?

6. **Commit**

F) Einfache Inventar-UI

`Zeitbudget: ca. 45 min`

Ziel: Ihr macht eure Inventar-App über eine einfache Oberfläche benutzbar.

1. **Inventar-Übersichtsseite**
   - Route `/inventory` verfügbar machen.
   - Zeigt dort mindestens eine Liste aller Devices mit
     * ihrem Typ
     * ihrem Standort
     * ihrem Status
   - Hinweis, ob das Gerät „frei“ oder „ausgeliehen“ ist.
2. **Formular „Gerät hinzufügen“**
   - Einfaches Formular zum Anlegen neuer Devices:
3. **Formular „Gerät ausgeben / zurückgeben“**
   - Dropdown für `person` und `device` (nur verfügbare Devices).
   - Button „Ausgeben“ → `POST /assignments`.
   - In der Liste der aktiven Ausleihen ein Button „Zurückgeben“ → `POST /assignments/{id}/return`.
4. **Commit**
   - Template- und Routing-Änderungen committen.

G) MQTT-Events für Ausleihen

`Zeitbudget: ca. 20 min`

Ziel: Ihr nutzt MQTT jetzt in eurer eigenen Domäne.

1. **Überlegt** euch **sinnvolle Topics**, z.B.:
   - `inventory/assignments/returned`
   - `inventory/assignments/issued`
2. **Sendet** bei einer erfolgreichen Ausgabe oder Rückgabe ein MQTT-Event.
3. **Testet** das Verhalten **mit** einem **Subscriber**, z.B:

   ```
   mosquitto_sub -h localhost -t 'inventory/assignments/#' -v
   ```

4. **Prüft**, ob eure Events beim Ausführen eurer Flows sichtbar werden.
5. **Commit**

H) Prüfliste & Präsentation

Prüfliste am Ende

- [ ] `inventar-starter` wurde als Arbeitsbasis übernommen
- [ ] Ein sauberer Basisstand wurde zunächst auf `main` committed.
- [ ] die eigentliche Arbeit erfolgt in einem Feature-Branch.
- [ ] Init-SQL für Inventar ist angelegt und erzeugt alle Tabellen.
- [ ] REST-Endpunkte für Devices & Assignments funktionieren inkl. Domänenregeln.
- [ ] Mindestens eine einfache UI-Seite für Inventar ist vorhanden.
- [ ] MQTT wird für mindestens ein Inventar-Ereignis genutzt
- [ ] Ihr könnt Branch-Wechsel, Schema, API und einen UI-Flow erklären.

Ablauf in der Präsentation

1. Zeigt kurz eure laufende App.
2. Zeigt `/health`, `/docs` und eure Inventar-Seite.
3. Führt einen einfachen Flow vor:
   - Device anlegen
   - Device ausgeben
   - Device zurückgeben
4. Erklärt an 2–3 Stellen im Code:
   - eine Stelle im SQL-Schema
   - einen API-Endpunkt
   - eine umgesetzte Domänenregel